

Software Engineering Concept (SE LAB07)

Topic: Threads communication and synchronization

Lab Overview

The purpose of this lab is to deepen your understanding of multithreading and concurrency in Java through practical implementation of various concepts.

Task 1: Implement a Thread-Safe Bank Account

Create a BankAccount class that supports concurrent deposits and withdrawals without race conditions.

Step execution and approach:

- Running your code inside BankAccount.java
- Inside the code, we use synchronized to avoid race condition
- The purpose of synchronized is to allow 1 thread at a time
- if there are more thread incoming, they need to wait
- And it avoid thread run simultaneously which could cause a race condition

Output:

```
Thread-1 withdraw 600.0  
Thread-2 - Insufficient funds
```

Task 2: Producer-Consumer with Bounded Buffer

Implement a producer-consumer system where producers add items to a bounded buffer and consumers remove them.

Step execution and approach:

- This task contain 4 classes which are BoundedBuffer.java, Consumer.java, Producer.java and Main.java
- BoundedBuffer.java is for sharing the data between Producer and Consumer
- Producer.java is for produce the data
- Consumer.java is for consume or remove the data
- The capacity of the buffer contain only 5 slots
- When the producer create data it will add in the buffer if it not full and will awake the waiting threat (consumer thread or could be producer threat)
- For consumer, it will remove the data if the buffer has the data and will notify if there is an available slot

Output:

```
Producer-1 produced: 1  
Consumer-2 consumed: 1  
Producer-2 produced: 1  
Consumer-1 consumed: 1
```

```
Producer-2 produced: 2
Producer-1 produced: 2
Consumer-2 consumed: 2
Consumer-1 consumed: 2
Producer-2 produced: 3
Producer-1 produced: 3
Producer-1 produced: 4
Producer-2 produced: 4
Consumer-1 consumed: 3
Consumer-2 consumed: 3
Producer-1 produced: 5
Producer-2 produced: 5
Consumer-1 consumed: 4
Consumer-2 consumed: 4
Producer-1 produced: 6
Producer-2 produced: 6
Producer-1 produced: 7
Consumer-1 consumed: 5
Producer-2 produced: 7
Consumer-2 consumed: 5
Producer-1 produced: 8
Consumer-1 consumed: 6
Producer-2 produced: 8
Consumer-2 consumed: 6
Producer-1 produced: 9
Consumer-1 consumed: 7
Producer-2 produced: 9
Consumer-2 consumed: 7
Producer-1 produced: 10
Consumer-1 consumed: 8
Producer-2 produced: 10
Consumer-2 consumed: 8
Consumer-1 consumed: 9
Consumer-2 consumed: 9
Consumer-1 consumed: 10
Consumer-2 consumed: 10
```

Task 3: Reader-Writer Problem

Implement a solution where multiple readers can access a resource simultaneously, but writers need exclusive access.

Step execution and approach:

- Running your code in ReadWrite.java
- The objective is allow reader thread run simultaneously, but writer thread need exclusive access
- In the code, we implement Reentrant Lock which it will be a shared resource
- All the reader thread run simultaneously, when it passes through `rwLock.readLock().lock();`
- it will requesting permission to read a shared resource safely using a read lock
- all the reader hold the lock
- If a writer holds the write lock, this thread waits until the writer releases the lock

- Same for writer thread but the differences is reader run simultaneously while write need only exclusive access

Output:

Reading: Initial Data

Reading: Initial Data

Reading: Initial Data

Writing: Updated Data

Dining Philosophers Problem

Implement the classic Dining Philosophers problem:

- 5 philosophers sitting around a circular table
- 5 forks (one between each pair of philosophers)
- Each philosopher alternates between thinking and eating
- To eat, a philosopher needs both left and right forks
- Prevent deadlock and starvation Requirements:
- Each philosopher should eat at least 3 times
- Use proper synchronization to avoid deadlock
- Display philosopher states (thinking/hungry/eating)
- Implement timeout mechanism to break potential deadlocks
- Run simulation for 2 minutes and show statistics

Step execution and approach:

- Running your code in DiningPhilosophers.java
- Initialize forks, Initialize philosophers
- Philosopher thread execution (run() method)
- Main thread waits and Stop simulation
- Program ends

Output:

Philosopher 1 is THINKING

Philosopher 4 is THINKING

Philosopher 2 is THINKING

Philosopher 0 is THINKING

Philosopher 3 is THINKING

```
Philosopher 2 is HUNGRY
Philosopher 2 is EATING (1)
Philosopher 0 is HUNGRY
Philosopher 0 is EATING (1)
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 4 is HUNGRY
Philosopher 2 is THINKING
Philosopher 2 is HUNGRY
Philosopher 1 is THINKING
Philosopher 4 is THINKING
Philosopher 3 is EATING (1)
Philosopher 4 is HUNGRY
Philosopher 0 is THINKING
Philosopher 2 is THINKING
Philosopher 2 is HUNGRY
Philosopher 4 is THINKING
Philosopher 1 is HUNGRY
Philosopher 3 is THINKING
Philosopher 2 is EATING (2)
Philosopher 3 is HUNGRY
Philosopher 0 is HUNGRY
Philosopher 1 is THINKING
Philosopher 0 is EATING (2)
Philosopher 3 is THINKING
Philosopher 0 is THINKING
Philosopher 2 is THINKING
Philosopher 4 is HUNGRY
Philosopher 4 is EATING (1)
Philosopher 2 is HUNGRY
Philosopher 2 is EATING (3)
Philosopher 0 is HUNGRY
Philosopher 2 finished eating 3 times
Philosopher 0 is THINKING
Philosopher 0 is HUNGRY
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (1)
Philosopher 3 is EATING (2)
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 1 is THINKING
Philosopher 0 is EATING (2)
Philosopher 3 is THINKING
Philosopher 0 is THINKING
Philosopher 2 is THINKING
Philosopher 4 is HUNGRY
Philosopher 4 is EATING (1)
Philosopher 2 is HUNGRY
Philosopher 2 is EATING (3)
Philosopher 0 is HUNGRY
Philosopher 2 finished eating 3 times
Philosopher 0 is THINKING
Philosopher 0 is HUNGRY
```

```
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (1)
Philosopher 3 is EATING (2)
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 4 is HUNGRY
Philosopher 4 is EATING (1)
Philosopher 2 is HUNGRY
Philosopher 2 is EATING (3)
Philosopher 0 is HUNGRY
Philosopher 2 finished eating 3 times
Philosopher 0 is THINKING
Philosopher 0 is HUNGRY
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (1)
Philosopher 3 is EATING (2)
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 0 is HUNGRY
Philosopher 2 finished eating 3 times
Philosopher 0 is THINKING
Philosopher 0 is HUNGRY
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (1)
Philosopher 3 is EATING (2)
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 2 finished eating 3 times
Philosopher 0 is THINKING
Philosopher 0 is HUNGRY
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (1)
Philosopher 3 is EATING (2)
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (1)
Philosopher 3 is EATING (2)
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 4 is THINKING
Philosopher 4 is HUNGRY
Philosopher 4 is HUNGRY
Philosopher 1 is THINKING
Philosopher 0 is EATING (3)
Philosopher 3 is THINKING
Philosopher 0 finished eating 3 times
Philosopher 4 is EATING (2)
Philosopher 3 is HUNGRY
```

```
Philosopher 3 is THINKING
Philosopher 0 finished eating 3 times
Philosopher 4 is EATING (2)
Philosopher 3 is HUNGRY
Philosopher 0 finished eating 3 times
Philosopher 4 is EATING (2)
Philosopher 3 is HUNGRY
Philosopher 4 is EATING (2)
Philosopher 3 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (3)
Philosopher 4 is HUNGRY
Philosopher 1 finished eating 3 times
Philosopher 4 is THINKING
Philosopher 3 finished eating 3 times
Philosopher 4 is HUNGRY
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (3)
Philosopher 4 is HUNGRY
Philosopher 1 finished eating 3 times
Philosopher 4 is THINKING
Philosopher 3 finished eating 3 times
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (3)
Philosopher 4 is HUNGRY
Philosopher 1 finished eating 3 times
Philosopher 4 is THINKING
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (3)
```

```
Philosopher 4 is HUNGRY
Philosopher 1 finished eating 3 times
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (3)
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 1 is EATING (2)
Philosopher 1 is EATING (2)
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 1 is THINKING
Philosopher 3 is HUNGRY
Philosopher 3 is EATING (3)
Philosopher 1 is HUNGRY
Philosopher 1 is EATING (3)
Philosopher 4 is HUNGRY
Philosopher 1 finished eating 3 times
Philosopher 4 is THINKING
Philosopher 3 finished eating 3 times
Philosopher 4 is HUNGRY
Philosopher 4 is EATING (3)
Philosopher 4 finished eating 3 times
```